

4 Getting Started with Queries

29 May 2026 17:02

| 4 |

Getting Started with Queries

Chapter Overview

- » Query notation techniques
- » SQL query basics
- » Using an alias
- » Sorting results alphabetically
- » Limiting query results
- » Data Analysis Checkpoint

SQL is a powerful and robust language that offers the user a wide variety of commands. Although there are more commands than we could ever show in this book, there are a few simple and easy-to-use commands that we can start experimenting with immediately. This chapter will cover the basics of writing a good query and formatting the results. By the end of this chapter you will be able to select individual fields from a specific database and display those fields in alphabetical order. Let's get started!

Adding Comments to Queries

Before we begin composing our first SQL query, we will take a look at creating comments. Comments are plain English sentences used to help add insight and authoring information about SQL statements that you create. It

is considered an industry best practice to use comments in your SQL queries; they help both the SQL author and those that come after the author to quickly gain understanding and context regarding the intention and function of the given SQL statement.

There are two ways to create a comment. Preceding anything written in the Query Pane with two hyphens (--) creates a comment on that line. Our

example shows a comment created on line 1 (Figure 32).

A comment block is a multi-line comment. It is created using the front slash and star symbols to open the block /*, followed by a star and a front slash */, which closes the block. Anything that falls between the opening and closing symbols becomes part of the comment (Figure 33).



```
1  --This is a line of comment
2
3  SELECT
4      *
5  FROM
6      albums;
```

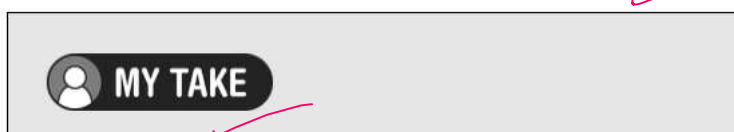
LINE OF COMMENT



COMMENT BLOCK

```
SQL 1
1  /*
2      CREATED BY: Walter Shields
3      CREATED ON: 3/13/2018
4      DESCRIPTION: This script selects all of the records from the albums table
5  */
6  SELECT
7      *
8  FROM
9      albums;
10
```

Our example shows some standard information that is very useful to include in a comment block. The author, date, and description provide significant value to anyone who encounters the SQL query or script.



Comments are often overlooked completely in SQL texts. We omit comments in future chapters for the sake of brevity only. When working with real-world databases, preexisting comments have saved me a lot of time that I would have otherwise spent writing additional queries to figure out how the database functioned. Comments become especially important when your queries will be read or used by others.

The Structure of a Basic Query

Writing a query is like asking a question in any natural language. The phrasing matters, the details matter, and the order of the words matters. The more detailed our question is, the more precise our answer will be.

In the creation of an SQL query, we need to consider the following five questions:

1. What database are we speaking to?
2. What table within that database are we requesting data from?
3. What fields within that table are we interested in?
4. Do we want to exclude any data, filter or omit any range or time period?
5. In one concise sentence, what does our query do?

The purpose of the foregoing questions is to help us build a bridge between the natural language we use every day and the language of SQL. If you are employed as a data analyst, you will be asked normal questions from your boss or coworkers about the business. You will have to convert these plain-language questions into SQL statements in order to answer them. After you have received the results of your query, you will then have to convert those results back into natural language that is accessible to all. Such is the nature of the job.

NOTE

If you are having trouble with a query, refer to the five questions above. Contemplating these questions will help you write your query.

Start Writing Your Query

To begin writing your first query, you can use the existing Execute SQL tab labeled "SQL 1" or start a new Query Pane tab, in the same way that you would open a new web browser tab. Click on the "Open Tab" icon.



With our new Query Pane opened, the first thing we will write is a comment block:

```
/*
  CREATED BY: <your name>
  CREATED ON: <date>
  DESCRIPTION: <A concise description of your
query,
i.e., question #5>
*/
```

After finishing our comment block, we can begin to write our first SQL statement. But to do that, we need a good question to answer. In chapter 2, we introduced the concept of an operational scenario for this book: imagining yourself as a data analyst for the fictional company sTunes. As you have seen from exploring the sTunes database in chapter 3, sTunes sells online music and has a digital library of artists, tracks, and albums and a list of customers who have purchased these songs. Let's say sTunes customer service wants to send a new promotional advertisement to all existing customers. Customer service wants to know if their customer contact list is

up to date, so they ask us if we can get a complete list of first names, last names, and customer email addresses (if available) from the database. How would we go about answering this question? Let's start by answering the questions listed earlier in the chapter.

1. What database are we speaking to?

In this case, we are dealing with only one database. The sTunes database should already be opened in DB Browser. If you have closed the browser since working through chapter 3, you may need to open it and the sTunes database file again.

2. What table within that database are we requesting data FROM?

We are looking for customer information. Browsing our Database Structure tab, we can see that we have a table named customers. That table looks promising.

3. What fields within that table are we SELECTing to display?

This question can be answered by our Browse Data tab. If we click on that tab and select the customers table in our drop-down menu, we see that it has fields for first name, last name, and email.

4. Do we want to exclude any data, filter or omit any range or time period?

In this case, sTunes customer service wants a list of all customers, so we'd better not omit anything.

5. In one concise sentence, what does our query do?

This query selects the first name, last name, and email address from the customers table.

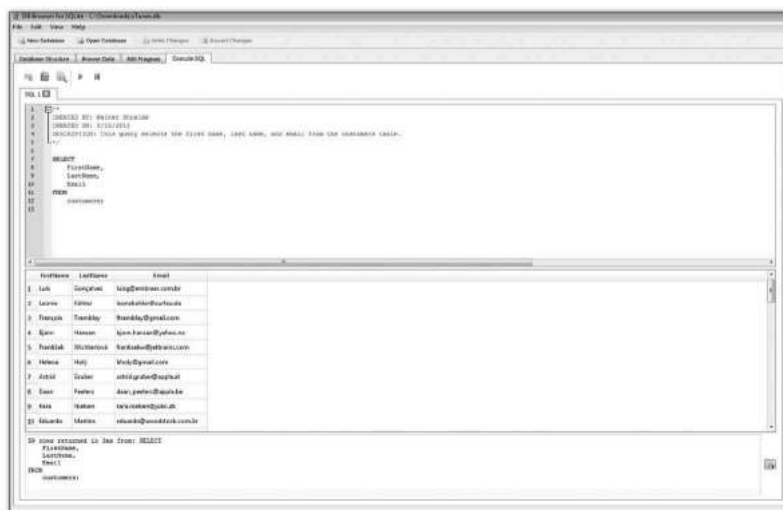
After adding your comment block, start by typing FROM customers.

This tells your query what table to look in for its data. Then type the keyword SELECT above the FROM clause, followed by the names of the fields within the customers table that you wish to view. Each field name is separated by a comma. The comma tells SQL to expect another field. The resulting code should look like this:

```
/*
  CREATED BY: Walter Shields
  CREATED ON: 3/13/2018
  DESCRIPTION: This query selects the first name,
  last name, and email from the customers table.
*/
```

```
SELECT
    FirstName,
    LastName,
    Email
FROM
    customers;
```

After composing this SQL query, run the statement by clicking on the Execute SQL play button in the menu bar. The results of the query are displayed in the Results Pane below (Figure 35). The Messages Pane also shows that the query returned 59 rows (aka records) in 3 milliseconds.



ON YOUR OWN:

»Add another field from the *customers* table to this query. Try adding the *Company* or *Phone* field to our *sTunes* mailing list. Don't forget to add an extra comma!

Coding Syntax Versus Coding Convention

As we mentioned in chapter 1, all queries must conform to a certain syntax in order to be understood by the SQL browser. When writing queries, there is much more to take into consideration than just making sure the SQL browser can understand you. It is also important that other database users can understand and follow your queries. The practice of writing queries in a standardized, readable, and consistent way is known as **coding convention**. Coding conventions vary in different database environments. This section will explain the coding conventions used in this book.

In earlier examples of queries, we used a * symbol after our SELECT keyword (in lieu of specifying individual fields). This special symbol tells the SQL browser to retrieve and display all fields in a table. Although this symbol is useful in some circumstances, in most cases it is a best practice to determine which fields you specifically want to select and call out those fields by name.

The semicolon at the end of the statement is optional in this case, since we are only writing one single SQL statement. The semicolon denotes the end of an SQL statement. Since most of the SQL queries we will be writing in this book are single statements, we will be omitting the semicolon from this point forward.

In the SELECT clause, we have chosen three fields to display. We must separate every field with a comma (except for after the last field). Omitting a comma between fields or adding a comma after the last field are both common syntax mistakes that will result in a syntax error appearing in your query Results Pane.

Notice that the code is broken up on multiple lines. We could write the entire query all on one line and the SQL browser would still recognize the code and return results. But it is a best practice to separate queries into

clauses with the content of each clause indented on a new line. Later in this text, our queries will become much longer and will contain many more clauses. Organizing our queries with indents and spacing increases readability and helps others to follow along.



REMEMBER

A clause is a subsection of an SQL statement that starts with a special SQL keyword (SELECT, FROM, etc.) and may include additional parameters and operators.

→ Adding an Alias to Your Columns

Often the technical language of the database will differ from the common language of a business. Sometimes you will be working with old databases or those that have not had their field names updated in a while. If you apply an *alias* to a field name (in order to describe the data in a way your coworkers are more likely to understand), any report you deliver will make much more sense to both you and anyone else reading along. Using aliases is also helpful if you want to clean up the column names and make your query output more readable or aesthetically pleasing.

In the following example, we will demonstrate several different ways to create an alias for selected field names from our *customers* table. An alias is always listed directly after the name of a field from our database. Aliases are commonly associated with the AS keyword, however, the use of the AS keyword between the field name and the alias name is optional in most RDBMS implementations.

```
/*
CREATED BY: Walter Shields
CREATED ON: 3/13/2018
DESCRIPTION: This query selects the first name,
last name, email, and phone number fields from
the customers table and demonstrates four
different ways to create an alias.
*/
```

```
SELECT
    FirstName AS 'First Name',
    LastName AS [Last Name],
    Email AS EMAIL,
    Phone CELL
FROM
    customers
```

As seen in the preceding query, we used the AS keyword for the first three fields, then omitted this keyword for the Phone field, which we renamed to CELL. If the alias you create contains multiple words (such as First Name and Last Name), it needs to be surrounded with some sort of demarcation, in this case either single quotes " or square brackets [] as shown. Since the aliases EMAIL and CELL are single words, they don't need any quotations or brackets.



SQL allows a great deal of variation in alias syntax. Other RDBMSs may not recognize every aliasing method listed here. If you get a syntax error when running your query, check to see how you designated your alias.



WITHOUT ALIASES

	FirstName	LastName	Email
1	Luis	Gonçalves	luisg@embraer.com.br
2	Leonie	Köhler	leonekohler@surfeu.de
3	François	Tremblay	ftremblay@gmail.com
4	Björn	Hansen	bjorn.hansen@yahoo.no
5	František	Wichterlová	frantisekw@jetbrains.com
6	Helena	Holy	hholy@gmail.com
7	Astrid	Gruber	astrid.gruber@apple.at
8	Daan	Peeters	daan_peeters@apple.be
9	Kara	Nielsen	kara.nielsen@jubii.dk
10	Eduardo	Martins	eduardo@woodstock.com.br

WITH ALIASES

	First Name	Last Name	EMAIL
1	Luis	Gonçalves	luisg@embraer.com.br
2	Leonie	Köhler	leonekohler@surfeu.de
3	François	Tremblay	ftremblay@gmail.com
4	Björn	Hansen	bjorn.hansen@yahoo.no
5	František	Wichterlová	frantisekw@jetbrains.com
6	Helena	Holy	hholy@gmail.com
7	Astrid	Gruber	astrid.gruber@apple.at
8	Daan	Peeters	daan_peeters@apple.be
9	Kara	Nielsen	kara.nielsen@jubii.dk
10	Eduardo	Martins	eduardo@woodstock.com.br

As you can see in Figure 36, the output on the left leaves the field names unchanged. The output on the right shows how our aliases modified the column names. Adding an alias will not change the data in the database. Aliases only alter how fields are displayed in the Results Pane.

ON YOUR OWN:

- » Add another field to this query and give it an alias.
- » Practice changing the alias syntax and omitting the AS keyword entirely. Does this have any effect on your output?



Avoid using any existing SQL keywords as your alias names. Using existing keywords could cause confusion, generate syntax errors, or cause the RDBMS to interpret your alias as a command instead.

Using the ORDER BY Clause

In our fictional scenario, we are providing a list of customers to sTunes customer service. It might be useful to order our results by the last name of our customers. To sort our results by customer last name, we need to use a new clause after our FROM clause. The ORDER BY clause allows us to sort our query results by any field(s) we choose. The default sort order is ascending (A to Z). The special keyword ASC, which specifies ascending order, is optional. To sort in descending order (Z to A), we would add the keyword DESC after the field being sorted. ORDER BY LastName DESC would sort the aliased column FirstName in descending order.

/*
 CREATED BY: Walter Shields
 CREATED ON: 3/13/2018
 DESCRIPTION: This query selects the first name,
 last name, and email from the customers table,
 ordered by Last Name.
 */

```
SELECT
    FirstName AS [First Name],
    LastName AS [Last Name],
    Email AS [EMAIL]
FROM
    customers
ORDER BY
    LastName ASC
```

 **IMAGE** fig. 37



SQL>
 EXECUTE AS (First Name),
 LastName AS (Last Name),
 Email AS (Email))
 FROM
 customers
 ORDER BY
 LastName ASC

First Name	Last Name	EMAIL
1. Roberto	Roberto	roberto.roberto@roberto.com
2. John	Roberto	john.roberto@roberto.com
3. Carlos	Roberto	carlos.roberto@roberto.com
4. Michael	Roberto	michael.roberto@roberto.com
5. Robert	Roberto	robert.roberto@roberto.com
6. Kelly	Chen	kelly.chen@roberto.com
7. Robert	Livingston	robert.livingston@roberto.com
8. Mary	Duke	mary.duke@roberto.com
9. John	Fernandez	john.fernandez@roberto.com
10. Edward	Francis	edward.francis@roberto.com

MS SQL Server 2018 (64-bit)
 Copyright (c) Microsoft Corporation
 All rights reserved.
 Microsoft SQL Server 2018 (64-bit)
 Copyright (c) Microsoft Corporation
 All rights reserved.



Without ORDER BY, every query will return data in the order that it was initially saved in the table.

We can use the ORDER BY clause to sort by multiple columns as well. In this example, we will sort by first name (ascending), and then by last name (descending). This will require us to list two fields in our ORDER BY clause. Just like with the SELECT clause, when listing multiple fields, we must separate them by commas.

```
/*
CREATED BY: Walter Shields
CREATED ON: 3/13/2018
DESCRIPTION: This query selects the first name,
last name, and email from the customers table,
ordered by first name (ascending), then last
name (descending).
*/
```

```
SELECT
    FirstName AS [First Name],
    LastName AS [Last Name],
    Email AS [EMAIL]
FROM
    customers
ORDER BY
    FirstName ASC,
    LastName DESC
```



The screenshot shows the SQL Developer interface with a query window and a results grid.

Query Window:

```

1 SELECT * FROM Users
2 WHERE ID = 1234567
3
4 --NOTE: This query returns the First Name, Last Name, and Email from the database table, ordered by First Name (ascending), then Last Name (descending).
5 --
6
7 SELECT
8     Firstname AS [First Name],
9     Lastname AS [Last Name],
10    Email AS [Email]
11 FROM
12     customers
13 ORDER BY
14     Firstname ASC,
15     Lastname DESC
16
17
18

```

Results Grid:

First Name	Last Name	Email
Adam	Smith	adamsmith@yahoo.com
Alexander	Smith	alexsmith@comcast
Alexis	Green	alexisgreen@yahoo.com
Egan	Hansen	egan.hansen@yahoo.com
Carroll	Perkins	carrollperkins@yahoo.com
Emily	Perkins	emilyp@comcast.net
Eric	Perkins	ericperkins@yahoo.com
Eric	Perkins	ericperkins@comcast.net
David	Green	davidgreen@yahoo.com
Christopher	Labadie	christopherlabadie@yahoo.com
Elizabeth	Martinez	elizabethmartinez@comcast.net

SQL Window:

```

18 view returned in the time: SELECT
19 Firstname AS [First Name],
20 Lastname AS [Last Name],
21 Email AS [Email]
22 ORDER BY
23     Firstname ASC,
24     Lastname DESC

```

When we run this query and look at the customer first names starting with D, we can see that first names are in ascending order and last names are in descending order.



If you use the **ORDER BY** statement on a column where some of the corresponding rows have empty values, you will see those values show up as NULL at the very top if you are using ascending order.

ON YOUR OWN:

»Rearrange the fields in the SELECT clause so that LastName is the first column instead of the second. Order the output by LastName. Does this make the list more readable?

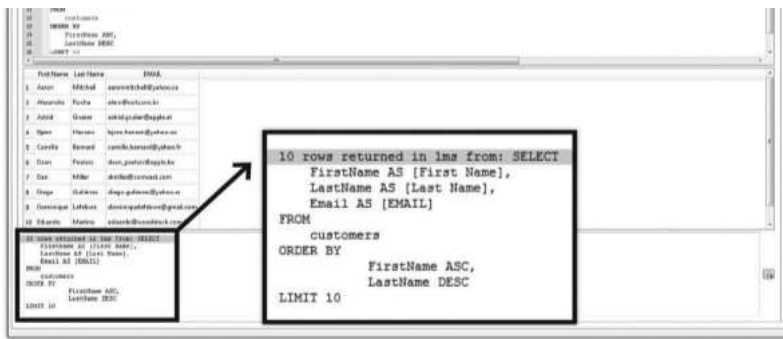
Selecting the Top Ten Records Using LIMIT

In our queries so far, we have returned every record available from the *customers* table. Although we have limited our records to three fields, and sorted those fields, we can see in our Messages Pane that we are returning 59 rows every time. If we are not interested in seeing all 59 records, we can limit our results to a specified number of rows. This is often helpful when sorting queries by a number (which we will demonstrate later), like highest price or largest sale. Adding the keywords **LIMIT 10** after the **ORDER BY** clause returns only the first ten records from the query in the sort order you specify. This number can be changed to any number you wish to limit your results by (provided there are at least that many records in the table).

```
/*
CREATED BY: Walter Shields
CREATED ON: 3/13/2018
DESCRIPTION: This query selects the first 10
records from the customers table, ordered by
first name (ascending), then last name
(descending).
*/
```

```
SELECT
    FirstName AS [First Name],
    LastName AS [Last Name],
    Email AS [EMAIL]
FROM
    customers
ORDER BY
    FirstName ASC,
    LastName DESC
LIMIT 10
```





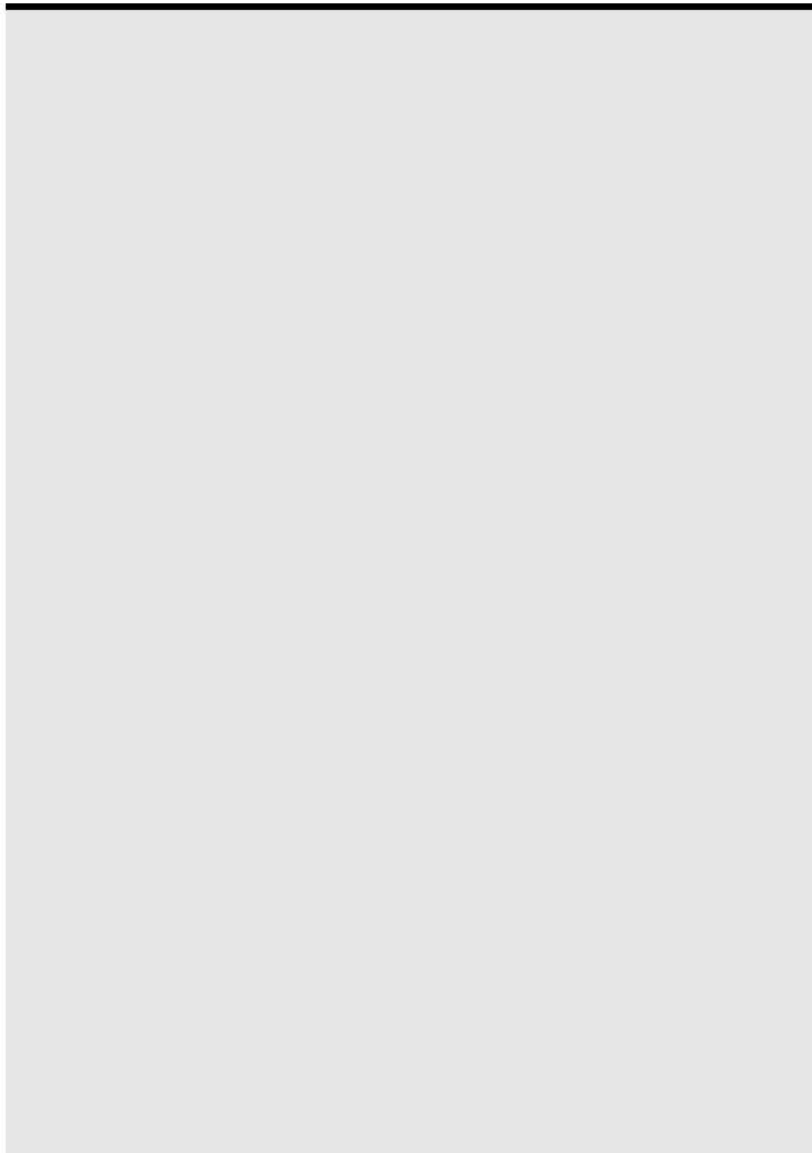
As we can see in Figure 39, our Messages Pane now says “10 rows returned in 1ms.” The query performs the ORDER BY operation first and then applies the limit.

NOTE

There is no requirement to use LIMIT with ORDER BY. In many cases, it makes sense to order your results by certain criteria before you limit them. If you do not use the ORDER BY clause, the results of the LIMIT statement will be returned to you in the order they were originally added to the table.

Data Analysis Checkpoint

1. How many customers' last names begin with B?
2. When sorted in descending order, which company appears at the top in the customers table?
3. How many customers do not have a postal code listed?



Chapter Recap

» Adding comments to queries is considered a best practice. A basic comment block can include your name, the date the query was created, and a brief description of what the query does.

» When composing a query, it is helpful to phrase what you want in a natural language first, then decide what keywords and clauses you need.

you need.

» A basic query usually begins with a SELECT clause that specifies what fields to display from the table specified in the FROM clause.

» An alias used after the AS keyword can rename or reformat the field names in your query results. This change is only cosmetic and does not affect the data in the database.

» Using ORDER BY allows you to sort alphabetically by field(s), either in ascending (ASC) or descending (DESC) order.

» You can limit the number of results returned by using LIMIT after your ORDER BY clause. You must include the number of results you wish to show.